

[C++, QT 2일차 과제 보고서]

이름 : 김도훈
학번: 2026406041

<목차>

- (1) 과제 1에 대한 설명
 - (1)-1 무엇을 만들었는지
 - (1)-2 어떻게 만들었는지, 왜 그렇게 만들었는지
 - (1)-3 실행 결과
- (2) 과제 2에 대한 설명
 - (2)-1 무엇을 만들었는지
 - (2)-2 어떻게 만들었는지, 왜 그렇게 만들었는지
 - (2)-3 실행 결과

1. 과제1

과제1은 QT의 기능들을 익히기 위해서 버튼, 슬라이더, 라벨 등을 사용해서 간단한 입출력 기능을 가진 로봇 컨트롤 패널을 제작하는 것이다. 예를 들어 전진 버튼을 눌렀을 때 로봇 상태를 표시해주는 라벨에 전진이라는 텍스트를 출력해 로봇이 현재 어떤 상태인지 표시해주는 기능 등이 있다.

1-(1) 무엇을 만들었는지

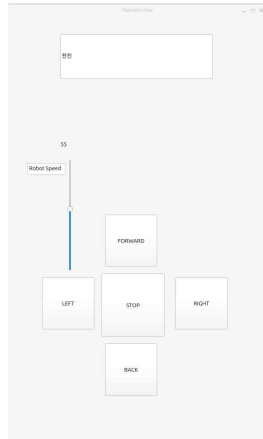
위에서 설명했듯이 이 과제1에서는 로봇의 상태를 입출력 해주는 로봇 컨트롤 패널을 만들었다.

1-(2) 어떻게 만들었는지, 왜 그렇게 만들었는지

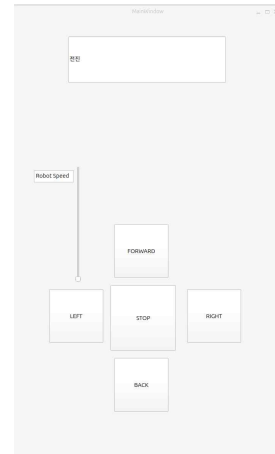
이 과제에서는 로봇 패널을 만든 것이기 때문에 ui 배치도 중요하다고 생각했다. 그래서 로봇 사용자가 로봇 패널을 사용해 로봇을 제어할 때 쉽게 사용할 수 있도록 우리가 사용하는 일반적인 리모콘과 게임 컨트롤러와 비슷하게 ui를 배치했다. 이것이 우리가 ui를 배우는 긍정적인 이유라고 생각한다. 배치 후에는 각 버튼에 go to slot을 선택하여 mainwindow.cpp에 버튼 함수들과 자동으로 signal, slot을 구현해 버튼을 클릭 했을 시 실행되는 함수 코드를 구현했다. 각 함수들에는 로봇의 상태를 출력할 수 있도록 ui -> QLineEdit -> setText("로봇의 상태") 코드를 사용하여 로봇의 전진 후진 우회전, 좌회전, 정지 등의 상태를 라벨에 출력할 수 있도록 구현했다.

그리고 버티컬 슬라이더를 사용해 로봇의 속도와 그 속도 값을 ui에 출력하는 기능을 구현했다. 슬라이더는 signal-slot으로 동작한다. connect(ui->slider, &QSlider::valueChanged, this, &MainWindow::onslideValueChanged); 이 connect() 함수를 사용해서 슬라이더 값이 바뀔 때 마다 발생하는 signal을 받고 MainWindow::onslideValueChanged 이 슬롯 함수가 실행되도록 해 슬라이더 값이 변할 때 Label에 슬라이더 값이 출력되도록 구현했다.

1-(3) 실행 결과



슬라이더 실행



버튼 실행

2. 과제2

과제2는 C++ 개념 중 vectot를 활용해 Queue 또는 stack을 활용해 UI를 제작하는 과제이다.

2-1 무엇을 만들었는지

나는 이 조건들을 모두 만족할 수 있는 프로그램이 무엇이 있을까? 고민하다가 stack 구조가 우리 일상생활에 적용하기 유용하다고 생각했고 stack 구조를 사용하기로 했다. stack의 후입 선출 구조를 어떻게 활용할지 고민했고 후입 선출은 어떤 과거의 동작을 다시 되돌리는 것에 사용될 수 있을 거 같다는 생각했다. 그래서 계산기를 사용할 때 과거의 기록들을 버튼 하나로 되돌리기 할 수 있는 기능을 만들게 되었다.

2-2 어떻게 만들었는지, 왜 그렇게 만들었는지

일단 계산기를 만들다 보니 계산기의 고유 기능인 연산을 가능하도록 하는 것이 우선이라고 생각해 덧셈 뺄셈 연산만 구현을 먼저 했다. UI에서 덧셈, 뺄셈 기능의 버튼을 만들었고 나중에 사용할 삭제 기능과 되돌리기 기능 버튼도 만들었다. 그리고 숫자를 입력할 수 있게 0부터 9까지의 버튼을 만들어서 숫자를 라벨에 입력할 수 있게 구현했다. 숫자를 입력하는 방식을 구현할 때 라벨을 사용해 노트북 키보드로 숫자를 입력하는 방식과 내가 구현한 방법 둘 중에 고민을 했다. 하지만 우리는 ui를 제작하고 있는 것이 때문에 숫자 입력 방식을 UI를 통해 하는 것이 적합하다고 생각해 이 방식을 선택했다. 이렇게 14개의 버튼들을 mainwindow.ui 파일에 의도에 맞게 배치를 했고 go to slot을 선택하여 자동으로 signal과 slot을 구현해 버튼과 mainwindow.cpp 파일의 함수와 연결시켰다. 그 후 버튼을 눌렀을 때 라벨에 버튼에 해당하는 텍스트가 누적되어 출력되도록 구현했다. 누적되는 것은 일반적으로 라벨에 텍스트를 출력하는 방식으로 출력해서는 안 된다. 그럼 입력할 때 마다 전에 입력했던 숫자 덮어진다. 이

렇게 `ui->lineEdit->setText(ui->lineEdit->text() + "1");` 출력했던 것에 + 연산자를 사용해 출력하는 방식을 사용해야 누적되도록 사용할 수 있다. 그리고 삭제 버튼을 눌렀을 때는 `clear()` 함수를 사용해서 라벨에 출력된 텍스트를 삭제하도록 했다. 다음 계산 결과를 보기 위해서 = 버튼을 눌렀을 때 현재 라벨에 출력되어 있는 문자열을 `ui->lineEdit->text()` 이 코드를 사용해 읽어오도록 했고 변수에 저장하도록 했고 읽어온 문자열을 문자 하나나 무엇인지 알기 위해서 `if (ch.isDigit())` 이 조건문을 사용하여 0부터 9까지의 숫자면 1을 연산 기호면 0을 반환하도록해 숫자와 연산자를 구분했다. 그 후 하위 중첩 조건문을 사용해

```
if (ch.isDigit()) {
    numBuffer += ch; // 숫자를 더하는게 아님 문자를 문자열 만들어주는 함수
} else if (ch == '+' || ch == '-') {
    if (!numBuffer.isEmpty()) {
        numbers.push_back(numBuffer.toInt());
        numBuffer.clear();
    }
    operators.push_back(ch);
}
```

1이 반환되면 숫자인 것을 판단하고 `numBuffer += ch`를 사용해 문자들을 문자열로 만들어 일의 자리 숫자, 이의 자리 숫자 이외에 자릿수도 연산이 가능하도록 했다. 그리고 이 숫자들은 자료형이 처음에는 문자, 문자열들이기 때문에 계산 사용할 수 있도록 `int` 형으로 변환하여 `vector`를 사용하여 메모리를 할당한 배열에 저장했다. 연산자도 마찬가지로 숫자가 아니면 연산자를 저장하는 배열에 저장했다. 그 후 반복문을 사용해서 두 배열에 인덱스에 접근해 값들을 가져와 계산하도록 했다. 숫자 계산이 기능이 구현 후에는 스택을 사용한 되돌리기 기능을 구현했다. 되돌리기 기능 구현의 핵심 아이디어도 위의 숫자 계산과 큰 차이가 없었다.

```
void MainWindow::on_pushButton_4_clicked() // Undo
{
    if (history.empty()) {
        return;
    }
    QString prev = history.back(); // 맨 뒤(가장 최근) 값을 꺼냄
    history.pop_back(); // 꺼낸 값은 벡터에서 제거
    ui->lineEdit->setText(prev); // 그 값으로 화면 되돌림
}
```

이 기능 또한 벡터로 만든 배열에 현재 라벨에 출력되고 있는 텍스트들을 배열 맨 뒤에 추가하는 것이다. 되돌리기 버튼을 눌렀을 때는 배열의 맨 뒤 값을 다시 라벨에 출력하게 하여 되돌리기 기능을 구현했다. 이 부분은 `push_back`, `back` 등 배열에 값들을 꺼내고 넣는 함수들을 구현할 때 LLM의 도움을 받아 구현했다.

2-3 실행 결과

